Data Analysis from Dbeaver – sql and python: Comparison:

SQLite Test.db

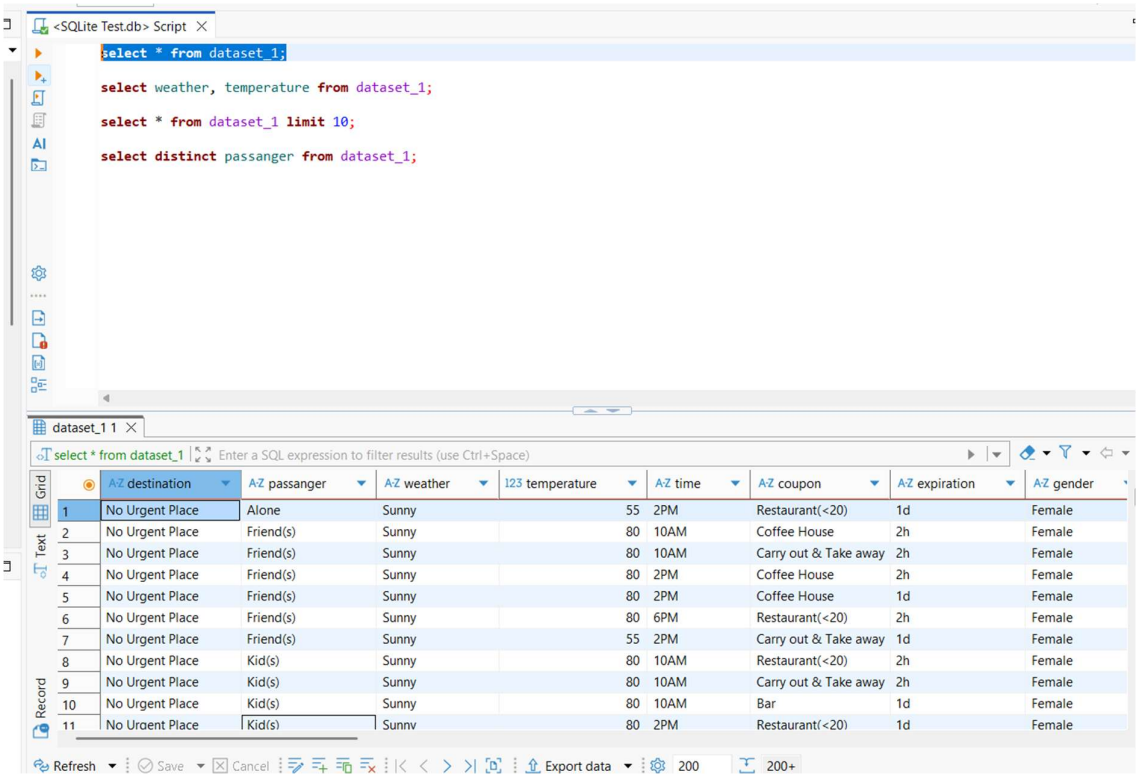
dataset_1_20251118107.csv

table_to_join_20251181809.csv

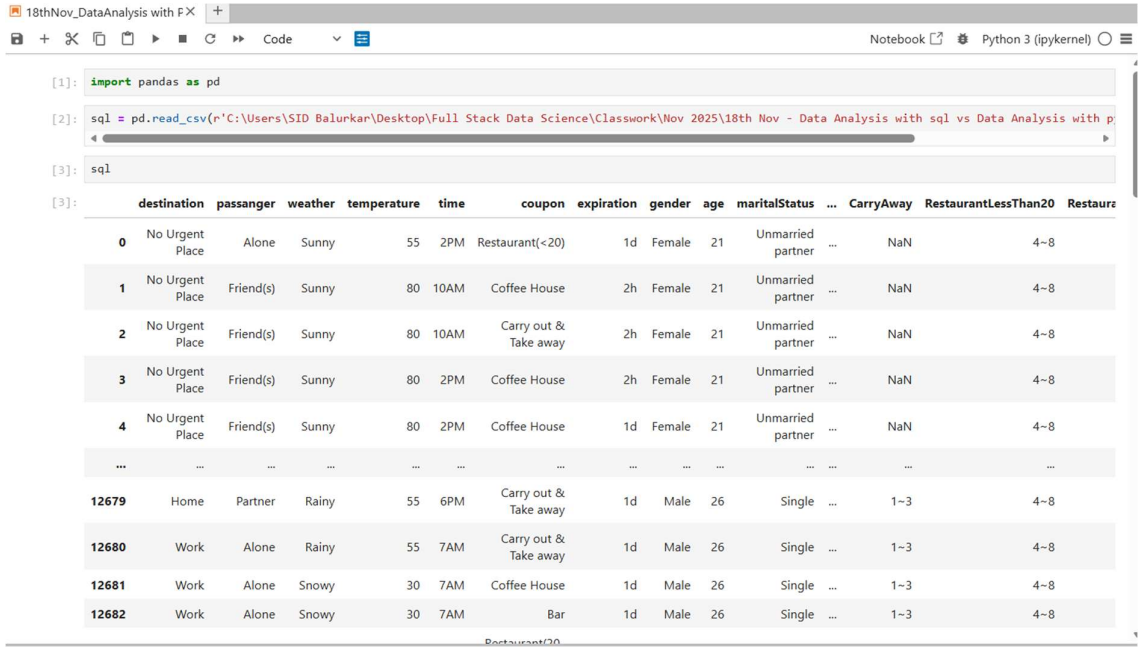
table_to_union_20251181809.csv

[In-Vehicle Coupon Recommendation - UCI Machine Learning Repository]

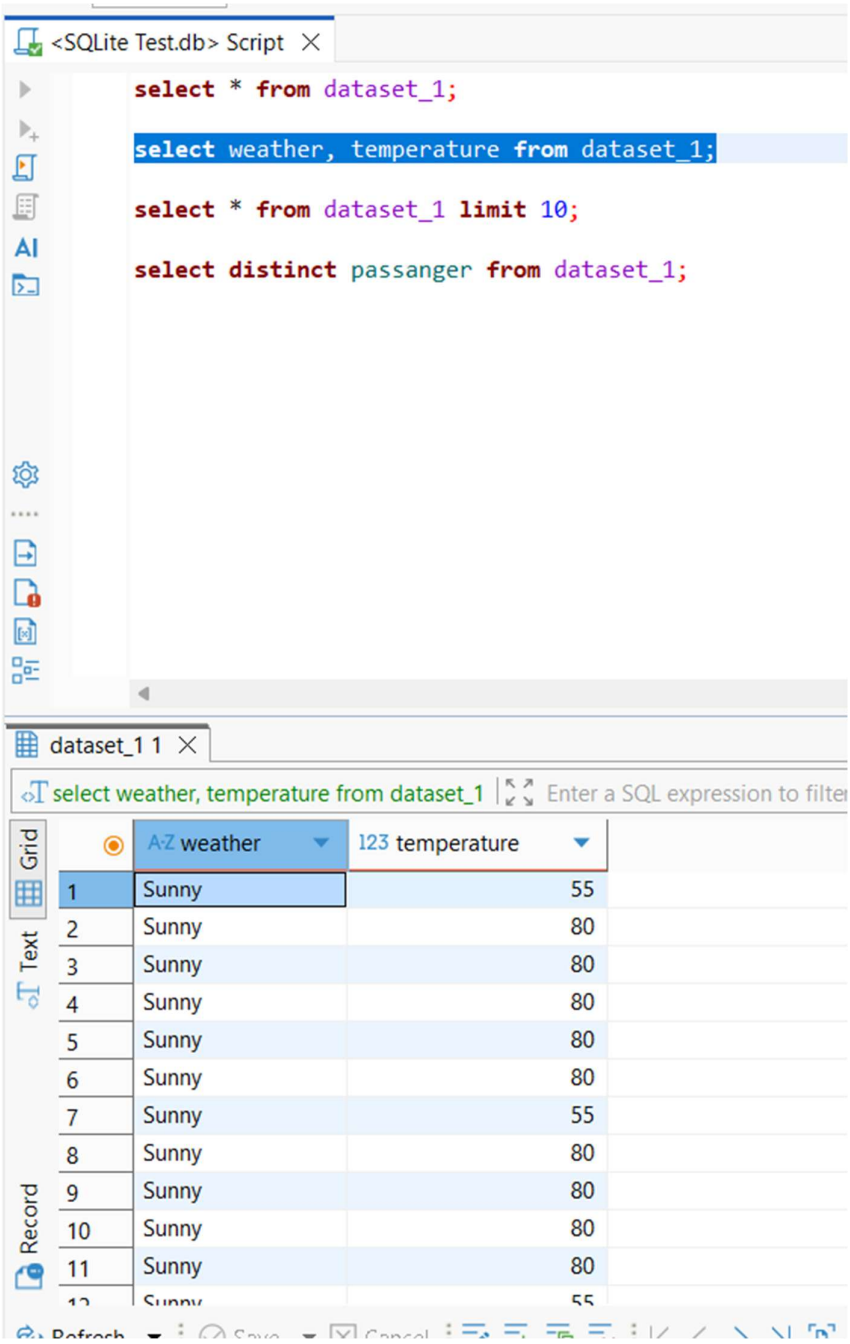
The csv files are exported from Dbeaver. Right click on table and export to csv and save and select

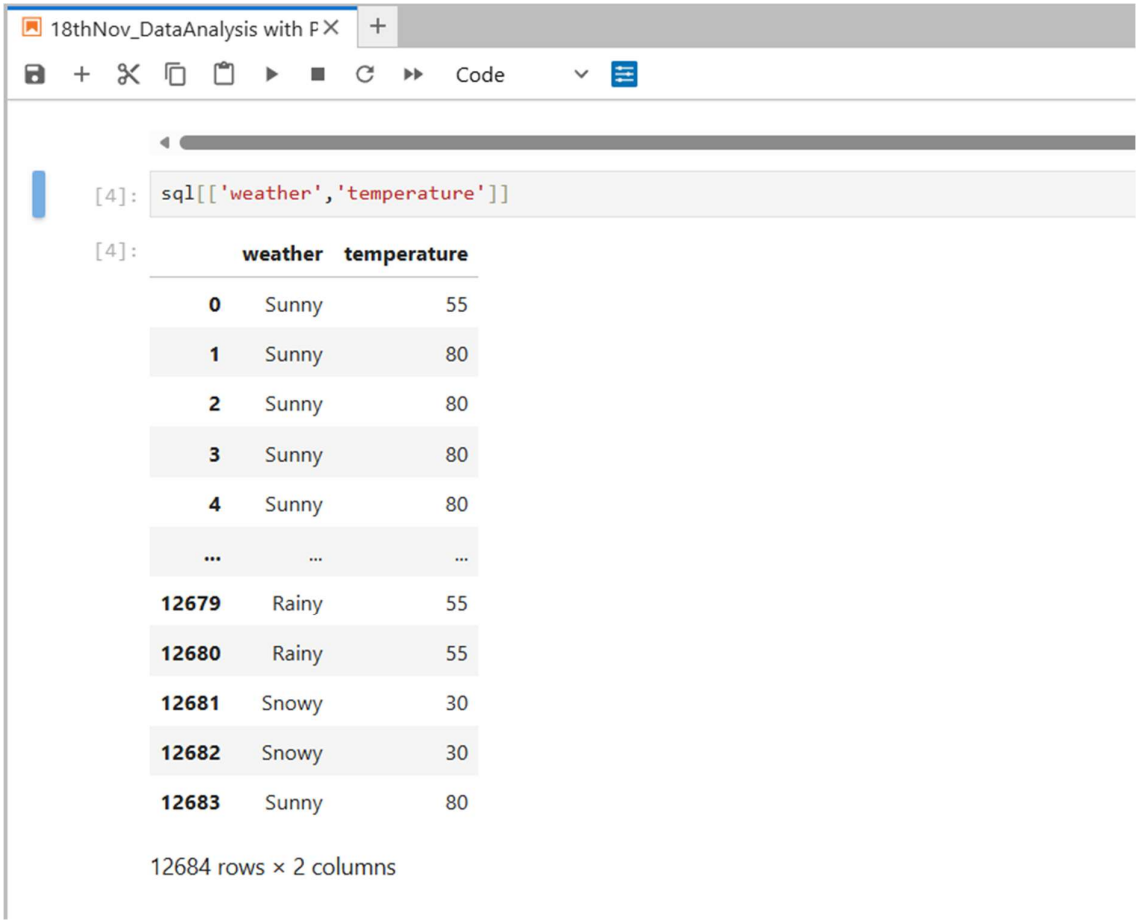


	destination	passanger	weather	temperature	time	coupon	expiration	gender
1	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female
3	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female
5	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female
6	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female
7	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female
11	No Urgent Place	Kid(s)	Sunny	80	2PM	Restaurant(<20)	1d	Female



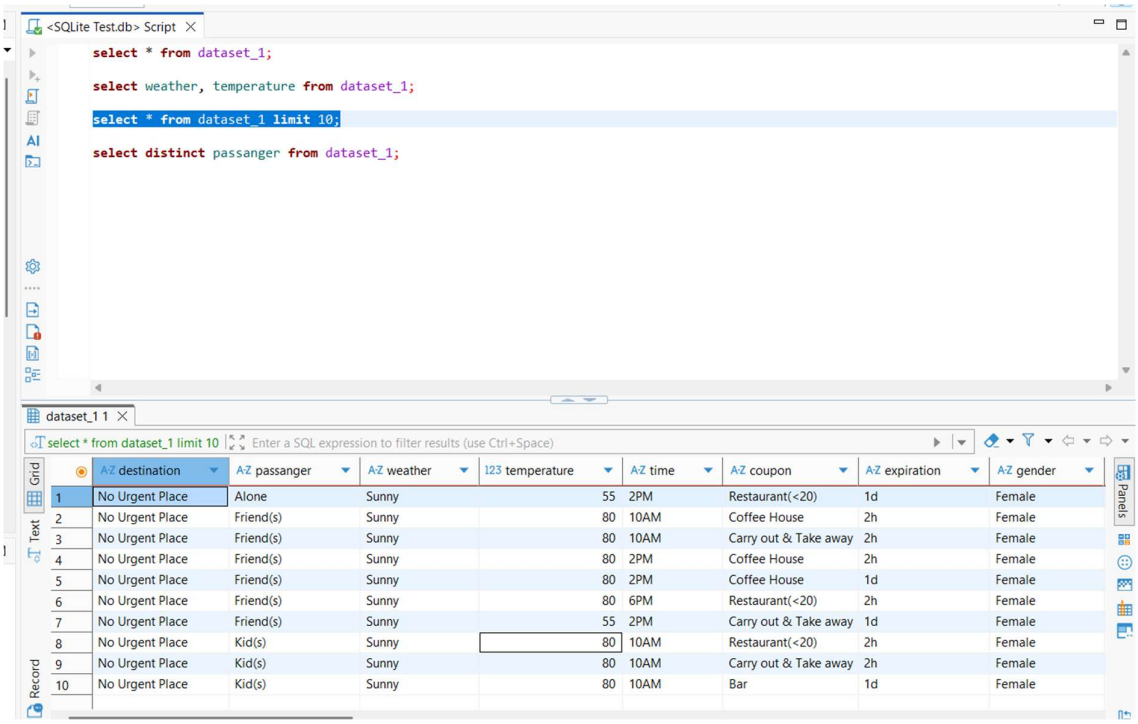
	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaura
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4~8	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4~8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	
12680	Work	Alone	Rainy	55	7AM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	
12681	Work	Alone	Snowy	30	7AM	Coffee House	1d	Male	26	Single	...	1~3	4~8	
12682	Work	Alone	Snowy	30	7AM	Bar	1d	Male	26	Single	...	1~3	4~8	


<SQLite Test.db> Script
select * from dataset_1;
select weather, temperature from dataset_1;
select * from dataset_1 limit 10;
select distinct passanger from dataset_1;
dataset_1 1
select weather, temperature from dataset_1
Enter a SQL expression to filter
Grid
Text
Record
A-Z weather
123 temperature
1 Sunny 55
2 Sunny 80
3 Sunny 80
4 Sunny 80
5 Sunny 80
6 Sunny 80
7 Sunny 55
8 Sunny 80
9 Sunny 80
10 Sunny 80
11 Sunny 80
Refresh
Save
Cancel

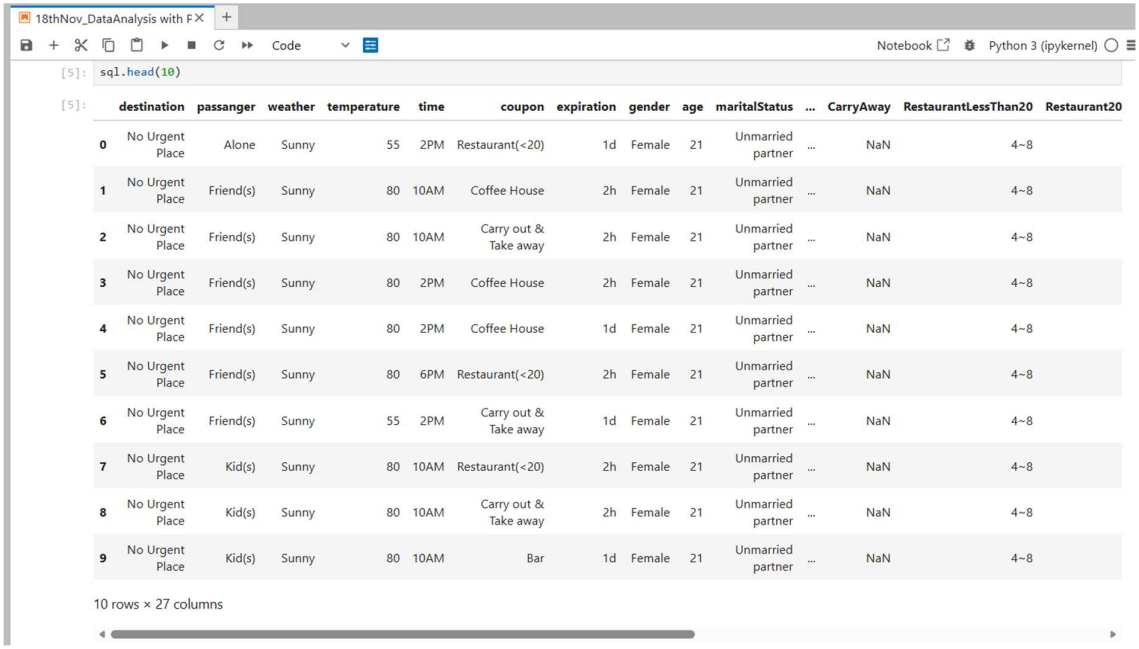


	weather	temperature
0	Sunny	55
1	Sunny	80
2	Sunny	80
3	Sunny	80
4	Sunny	80
...	...	...
12679	Rainy	55
12680	Rainy	55
12681	Snowy	30
12682	Snowy	30
12683	Sunny	80

12684 rows × 2 columns

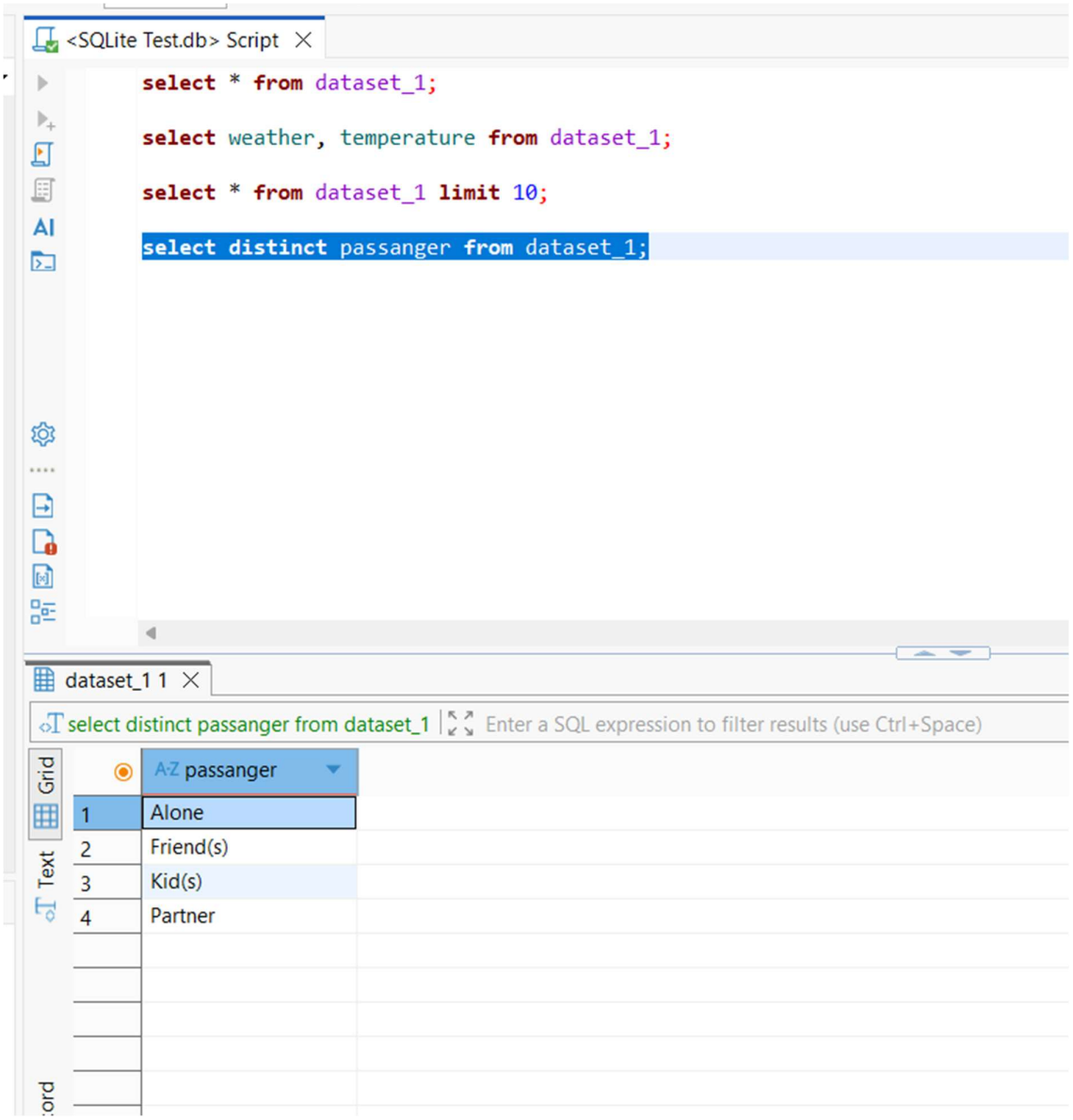


	destination	passanger	weather	temperature	time	coupon	expiration	gender
1	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female
3	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female
5	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female
6	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female
7	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female



	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant20
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4~8	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4~8	
5	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female	21	Unmarried partner	...	NaN	4~8	
6	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female	21	Unmarried partner	...	NaN	4~8	
7	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female	21	Unmarried partner	...	NaN	4~8	
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4~8	
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female	21	Unmarried partner	...	NaN	4~8	

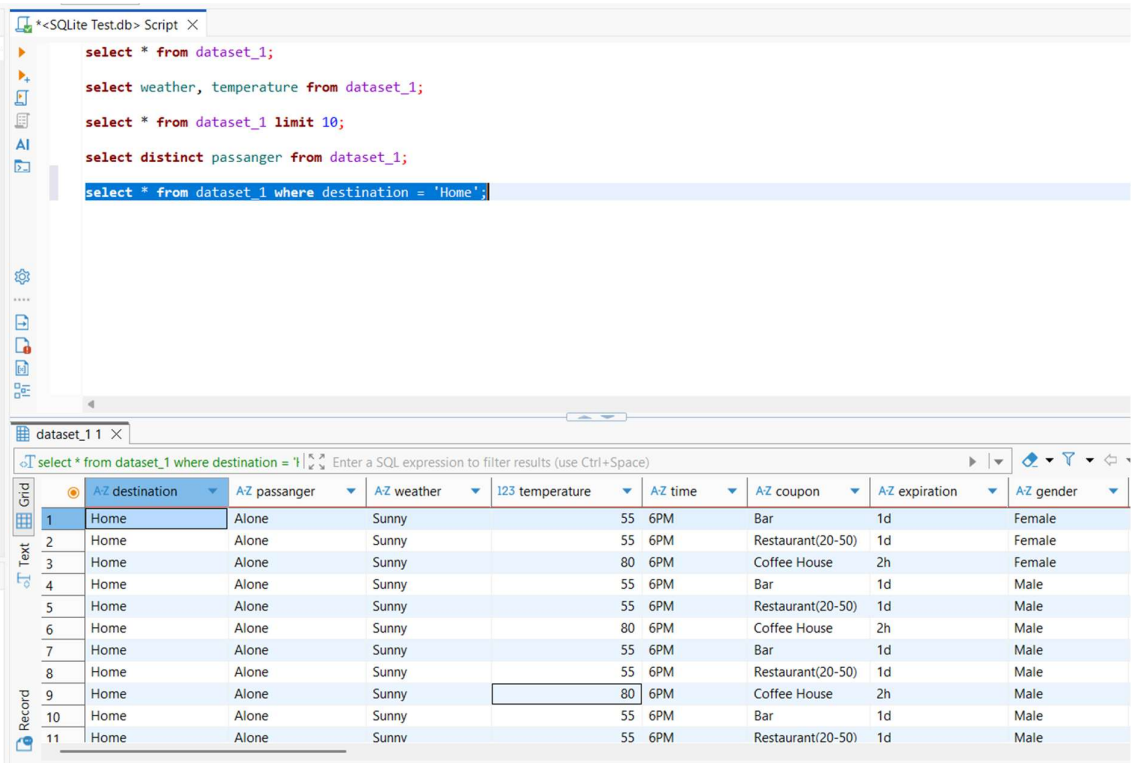
10 rows × 27 columns


<SQLite Test.db> Script
select * from dataset_1;
select weather, temperature from dataset_1;
select * from dataset_1 limit 10;
select distinct passanger from dataset_1;
dataset_1 1
select distinct passanger from dataset_1
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
passanger
1 Alone
2 Friend(s)
3 Kid(s)
4 Partner

```
[6]: sql['passanger'].unique
```

```
[6]: <bound method Series.unique of 0           Alone
     1         Friend(s)
     2         Friend(s)
     3         Friend(s)
     4         Friend(s)
               ...
     12679     Partner
     12680       Alone
     12681       Alone
     12682       Alone
     12683       Alone
     Name: passanger, Length: 12684, dtype: object>
```

```
[ ]:
```



	destination	passanger	weather	temperature	time	coupon	expiration	gender
1	Home	Alone	Sunny	55	6PM	Bar	1d	Female
2	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Female
3	Home	Alone	Sunny	80	6PM	Coffee House	2h	Female
4	Home	Alone	Sunny	55	6PM	Bar	1d	Male
5	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Male
6	Home	Alone	Sunny	80	6PM	Coffee House	2h	Male
7	Home	Alone	Sunny	55	6PM	Bar	1d	Male
8	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Male
9	Home	Alone	Sunny	80	6PM	Coffee House	2h	Male
10	Home	Alone	Sunny	55	6PM	Bar	1d	Male
11	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Male

18thNov_DataAnalysis with P

Code

Notebook Python 3 (ipykernel)

[10]:
```
sql[sql['destination'] == 'Home']
```

[10]:

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaura
13	Home	Alone	Sunny	55	6PM	Bar	1d	Female	21	Unmarried partner	...	NaN	4~8	
14	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Female	21	Unmarried partner	...	NaN	4~8	
15	Home	Alone	Sunny	80	6PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
35	Home	Alone	Sunny	55	6PM	Bar	1d	Male	21	Single	...	4~8	4~8	
36	Home	Alone	Sunny	55	6PM	Restaurant(20-50)	1d	Male	21	Single	...	4~8	4~8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
12675	Home	Alone	Snowy	30	10PM	Coffee House	2h	Male	26	Single	...	1~3	4~8	
12676	Home	Alone	Sunny	80	6PM	Restaurant(20-50)	1d	Male	26	Single	...	1~3	4~8	
12677	Home	Partner	Sunny	30	6PM	Restaurant(<20)	1d	Male	26	Single	...	1~3	4~8	
12678	Home	Partner	Sunny	30	10PM	Restaurant(<20)	2h	Male	26	Single	...	1~3	4~8	
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	

3237 rows × 27 columns

---

*<SQLite Test.db> Script

```
select * from dataset_1;

select weather, temperature from dataset_1;

select * from dataset_1 limit 10;

select distinct passanger from dataset_1;

select * from dataset_1 where destination = 'Home';

select * from dataset_1 order by coupon;
```

dataset_1 1

select * from dataset_1 order by coupon Enter a SQL expression to filter results (use Ctrl+Space)

Grid Text Record

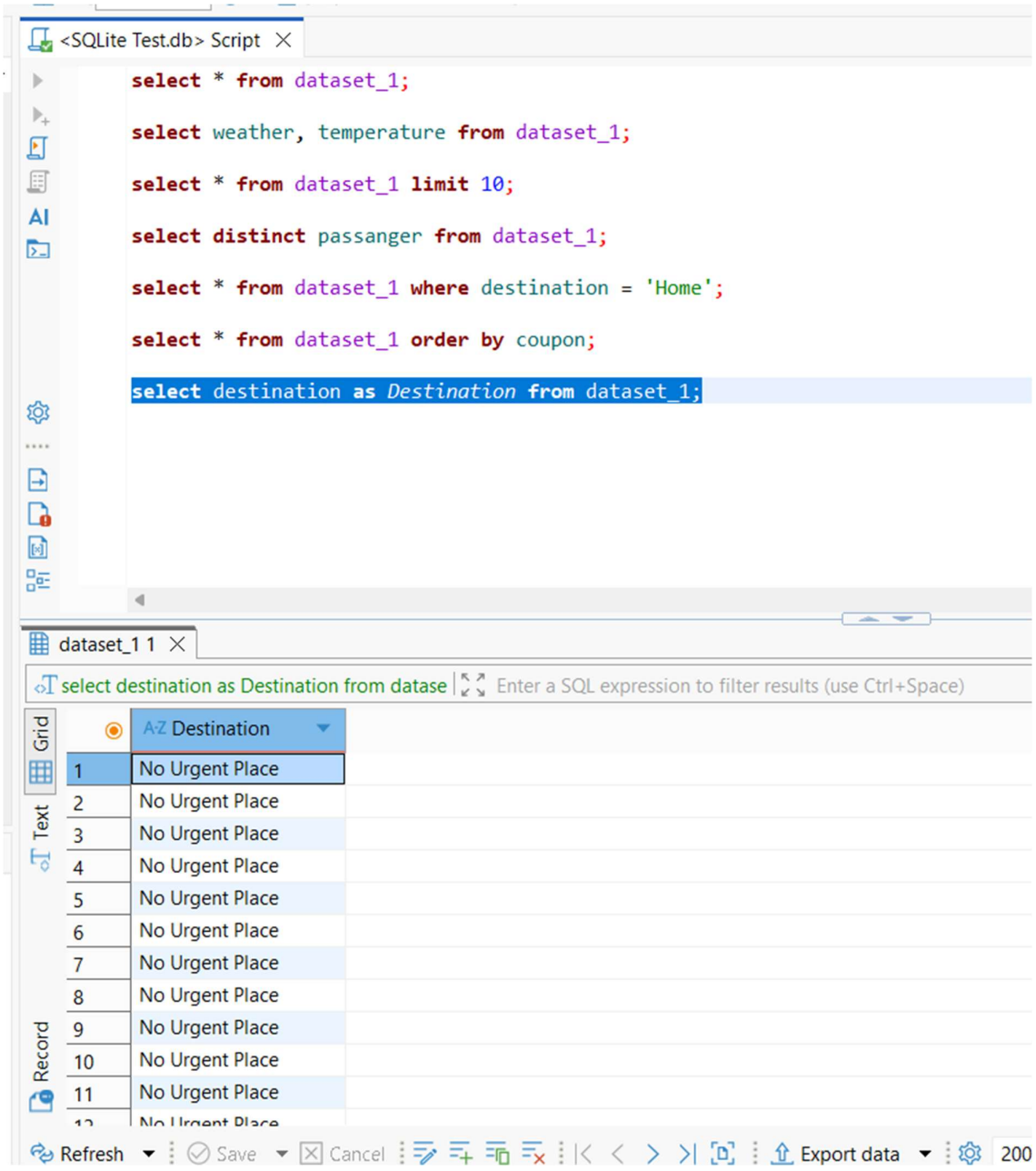
	destination	passanger	weather	temperature	time	coupon	expiration	gender
1	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female
2	Home	Alone	Sunny	55	6PM	Bar	1d	Female
3	Work	Alone	Sunny	55	7AM	Bar	1d	Female
4	No Urgent Place	Friend(s)	Sunny	80	10AM	Bar	1d	Male
5	Home	Alone	Sunny	55	6PM	Bar	1d	Male
6	Work	Alone	Sunny	55	7AM	Bar	1d	Male
7	No Urgent Place	Friend(s)	Sunny	80	10AM	Bar	1d	Male
8	Home	Alone	Sunny	55	6PM	Bar	1d	Male
9	Work	Alone	Sunny	55	7AM	Bar	1d	Male
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Male
11	Home	Alone	Sunny	55	6PM	Bar	1d	Male

[11]:
```
sql.sort_values('coupon')
```

[11]:

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restau
11702	Home	Partner	Sunny	30	10PM	Bar	2h	Female	50plus	Married partner	...	4~8	1~3	
9930	No Urgent Place	Alone	Snowy	30	2PM	Bar	1d	Female	21	Single	...	gt8	gt8	
10632	Home	Alone	Rainy	55	6PM	Bar	1d	Male	21	Single	...	gt8	less1	
7997	No Urgent Place	Friend(s)	Rainy	55	10PM	Bar	2h	Male	26	Unmarried partner	...	4~8	never	
11166	Work	Alone	Snowy	30	7AM	Bar	1d	Female	41	Married partner	...	gt8	1~3	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
10476	Home	Alone	Sunny	80	6PM	Restaurant(<20)	1d	Female	31	Unmarried partner	...	1~3	1~3	
5447	Home	Alone	Sunny	80	10PM	Restaurant(<20)	2h	Female	50plus	Single	...	less1	less1	
10478	Home	Alone	Snowy	30	10PM	Restaurant(<20)	2h	Female	31	Unmarried partner	...	1~3	1~3	
5440	No Urgent Place	Alone	Sunny	80	2PM	Restaurant(<20)	2h	Female	50plus	Single	...	less1	less1	
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	

12684 rows × 27 columns



`df.rename(columns={'destination': 'Destination'}, inplace=True)` is a **pandas** command used to rename a column in a DataFrame.

✔ What it does

It changes the column name `destination` to `Destination` (note the capital "D").

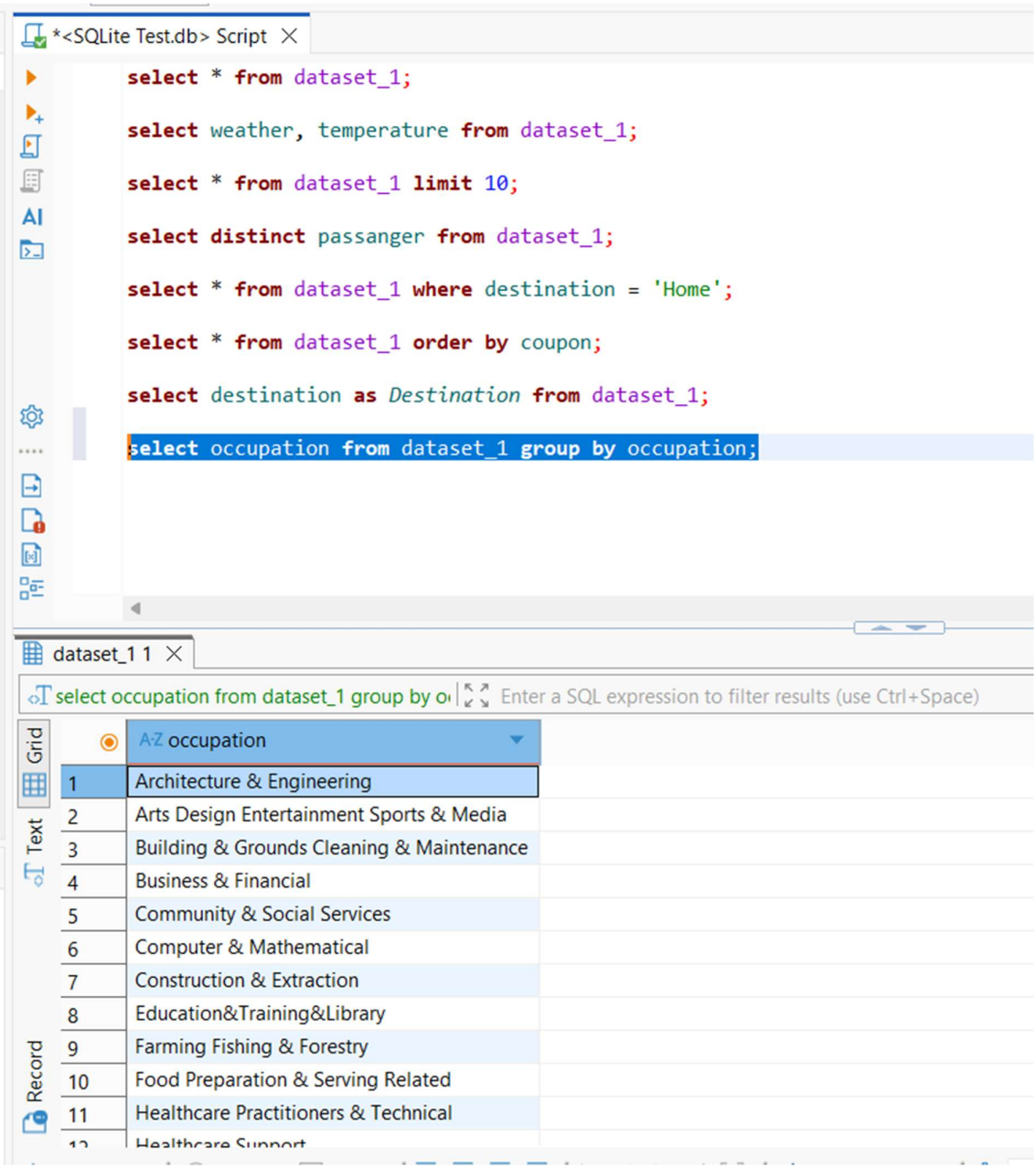
✔ Explanation of parameters:

- **columns={...}** → dictionary of old_name : new_name
- **inplace=True** → modifies the existing DataFrame directly (no need to assign back)

[16]:
```
sql.rename(columns={'destination':'Destination'},inplace = True)
sql['Destination']
```

[16]:
```
0        No Urgent Place
1        No Urgent Place
2        No Urgent Place
3        No Urgent Place
4        No Urgent Place
              ...
12679              Home
12680              Work
12681              Work
12682              Work
12683              Work
Name: Destination, Length: 12684, dtype: object
```

[ ]:


*<SQLite Test.db> Script
select * from dataset_1;
select weather, temperature from dataset_1;
select * from dataset_1 limit 10;
select distinct passanger from dataset_1;
select * from dataset_1 where destination = 'Home';
select * from dataset_1 order by coupon;
select destination as Destination from dataset_1;
select occupation from dataset_1 group by occupation;
dataset_1 1
select occupation from dataset_1 group by o
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
occupation
1 Architecture & Engineering
2 Arts Design Entertainment Sports & Media
3 Building & Grounds Cleaning & Maintenance
4 Business & Financial
5 Community & Social Services
6 Computer & Mathematical
7 Construction & Extraction
8 Education&Training&Library
9 Farming Fishing & Forestry
10 Food Preparation & Serving Related
11 Healthcare Practitioners & Technical

```python
df.groupby('occupation').size().to_frame('Count').reset_index()
```

Step-by-step explanation

1. `df.groupby('occupation')`
   Groups the DataFrame by the values in the **occupation** column.
2. `.size()`
   Counts the number of rows in each occupation group.
3. `.to_frame('Count')`
   Converts the resulting Series into a DataFrame and names the column "**Count**".
4. `.reset_index()`
   Resets the index so that *occupation* becomes a normal column instead of the index.

```
sql.groupby('occupation').size()
```

```
occupation
Architecture & Engineering                    175
Arts Design Entertainment Sports & Media      629
Building & Grounds Cleaning & Maintenance      44
Business & Financial                          544
Community & Social Services                   241
Computer & Mathematical                      1408
Construction & Extraction                     154
Education&Training&Library                    943
Farming Fishing & Forestry                     43
Food Preparation & Serving Related            298
Healthcare Practitioners & Technical          244
Healthcare Support                            242
Installation Maintenance & Repair             133
Legal                                         219
Life Physical Social Science                  170
Management                                    838
Office & Administrative Support               639
Personal Care & Service                       175
Production Occupations                        110
Protective Service                            175
Retired                                       495
Sales & Related                              1093
Student                                      1584
Transportation & Material Moving              218
Unemployed                                   1870
dtype: int64
```

[28]:
```
sql.groupby('occupation').size().to_frame('Count')
```

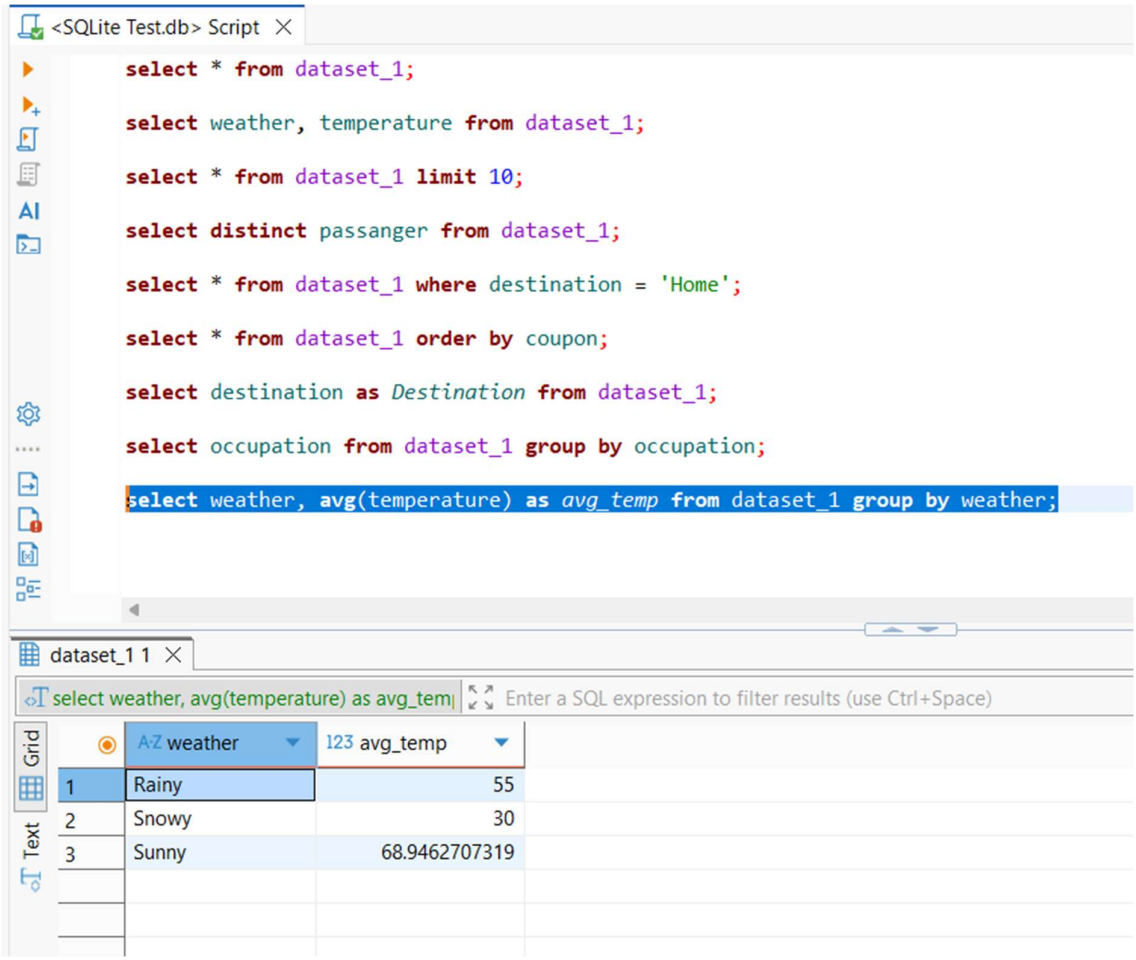
[28]:

occupation	Count
Architecture & Engineering	175
Arts Design Entertainment Sports & Media	629
Building & Grounds Cleaning & Maintenance	44
Business & Financial	544
Community & Social Services	241
Computer & Mathematical	1408
Construction & Extraction	154
Education&Training&Library	943
Farming Fishing & Forestry	43
Food Preparation & Serving Related	298
Healthcare Practitioners & Technical	244
Healthcare Support	242
Installation Maintenance & Repair	133
Legal	219
Life Physical Social Science	170
Management	838

[31]:
```
#select occupation from dataset_1 group by occupation;   -SQL
sql.groupby('occupation').size().to_frame('Count').reset_index()
```

[31]:

	occupation	Count
0	Architecture & Engineering	175
1	Arts Design Entertainment Sports & Media	629
2	Building & Grounds Cleaning & Maintenance	44
3	Business & Financial	544
4	Community & Social Services	241
5	Computer & Mathematical	1408
6	Construction & Extraction	154
7	Education&Training&Library	943
8	Farming Fishing & Forestry	43
9	Food Preparation & Serving Related	298
10	Healthcare Practitioners & Technical	244
11	Healthcare Support	242
12	Installation Maintenance & Repair	133
13	Legal	219
14	Life Physical Social Science	170
15	Management	838
16	Office & Administrative Support	639




```
[34]: sql.groupby('weather')['temperature'].mean()
```

```
[34]: weather
      Rainy    55.000000
      Snowy    30.000000
      Sunny    68.946271
      Name: temperature, dtype: float64
```

```
[35]: #select weather, avg(temperature) as avg_temp from dataset_1 group by weather;
      sql.groupby('weather')['temperature'].mean().to_frame('avg_temp').reset_index()
```

[35]:

	weather	avg_temp
0	Rainy	55.000000
1	Snowy	30.000000
2	Sunny	68.946271

```
select weather, count(temperature) as count_temp from dataset_1 group by weather;
```

dataset_1 1

select weather, count(temperature) as count_t | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	weather	count_temp
1	Rainy	1,210
2	Snowy	1,405
3	Sunny	10,069

[37]:
```
#select weather, count(temperature) as count_temp from dataset_1 group by weather;
sql.groupby('weather')['temperature'].size().to_frame('count_temp').reset_index()
```

[37]:

	weather	count_temp
0	Rainy	1210
1	Snowy	1405
2	Sunny	10069

[ ]:

```
select weather, count(distinct temperature) as count_distinct_temp from dataset_1 group by weather;
```

dataset_1 1

select weather, count(distinct temperature) a: | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	weather	count_distinct_temp
1	Rainy	1
2	Snowy	1
3	Sunny	3

[38]:
```
#select weather, count(distinct temperature) as count_distinct_temp from dataset_1 group by weather;
sql.groupby('weather')['temperature'].nunique().to_frame('count_distinct_temp').reset_index()
```

[38]:

	weather	count_distinct_temp
0	Rainy	1
1	Snowy	1
2	Sunny	3

```
select weather, count(distinct temperature) as count_distinct_temp from dataset_1 group
select weather, sum(temperature) as sum_temp from dataset_1 group by weather;
```

dataset_1 1

select weather, sum(temperature) as sum_tem | Enter a SQL expression to filter results (use Ctrl+Space)

	weather	sum_temp
1	Rainy	66,550
2	Snowy	42,150
3	Sunny	694,220

[39]:
```
#select weather, sum(temperature) as sum_temp from dataset_1 group by weather;
sql.groupby('weather')['temperature'].sum().to_frame('sum_temp').reset_index()
```

[39]:

	weather	sum_temp
0	Rainy	66550
1	Snowy	42150
2	Sunny	694220

```
select weather, min(temperature) as min_temp from dataset_1 group by weather;
```

dataset_1 1

select weather, min(temperature) as min_tem | Enter a SQL expression to filter results (use Ctrl+Space)

	weather	min_temp
1	Rainy	55
2	Snowy	30
3	Sunny	30

```
[40]: #select weather, min(temperature) as min_temp from dataset_1 group by weather;
      sql.groupby('weather')['temperature'].min().to_frame('min_temp').reset_index()
```

[40]:

	weather	min_temp
0	Rainy	55
1	Snowy	30
2	Sunny	30

```
select weather, max(temperature) as max_temp from dataset_1 group by weather;
```

dataset_1 1

select weather, max(temperature) as max_ten | Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z weather	123 max_temp
1	Rainy	55
2	Snowy	30
3	Sunny	80

```
[41]: #select weather, max(temperature) as max_temp from dataset_1 group by weather;
      sql.groupby('weather')['temperature'].max().to_frame('max_temp').reset_index()
```

[41]:

	weather	max_temp
0	Rainy	55
1	Snowy	30
2	Sunny	80

```
select occupation from dataset_1 group by occupation having occupation = 'Student';
```

dataset_1 1

select occupation from dataset_1 group by o | Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z occupation
1	Student

python

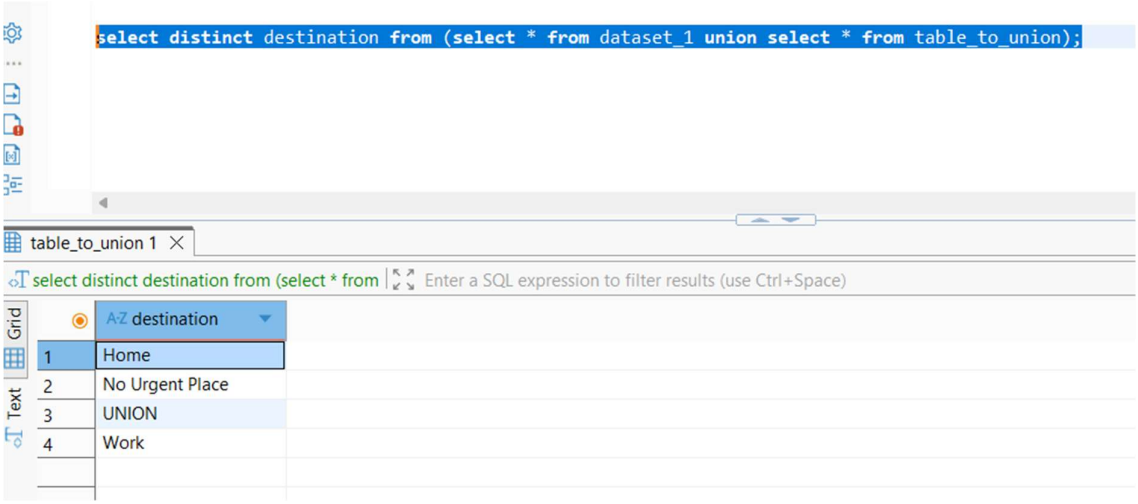
```python
df.groupby('occupation')
  .filter(lambda x: x['occupation'].iloc[0] == 'Student')
  .groupby('occupation')
  .size()
```

What it does

1. `df.groupby('occupation')`
   Groups the rows by occupation.
2. `.filter(lambda x: x['occupation'].iloc[0] == 'Student')`
   Keeps **only the groups where the occupation is exactly** `'Student'`.
   Since each group has the same occupation, this simply selects the **Student group**.
3. `.groupby('occupation')`
   Groups again (but now the DataFrame contains only `'Student'` rows).
4. `.size()`
   Counts how many rows are in the Student group.

```
: #select occupation from dataset_1 group by occupation having occupation = 'Student';
sql.groupby('occupation').filter(lambda x : x['occupation'].iloc[0] == 'Student').groupby('occupation').size()
```

```
: occupation
Student    1584
dtype: int64
```



```
[91]: #Read table_to_union
sql1 = pd.read_csv(r'C:\Users\SID Balurkar\Desktop\Full Stack Data Science\Classwork\Nov 2025\18th Nov - Data Analysis with sql vs Data Analysis with
sql1
```

[91]:

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant20
0	UNION	UNION	UNION	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	

1 rows × 27 columns

```
[93]: #SELECT DISTINCT destination FROM(SELECT * FROM dataset_1 UNION SELECT * FROM table_to_union)
pd.concat([sql,sql1])['Destination'].drop_duplicates()
```

```
[93]: 0     No Urgent Place
13              Home
16              Work
0                NaN
Name: Destination, dtype: object
```

```
select a.destination, a.time, b.part_of_day from dataset_1 a inner join table_to_join b on a.time = b.time;
```

dataset_1(+) 1

select a.destination, a.time, b.part_of_day fror | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text | Record

	destination	time	part_of_day
1	No Urgent Place	2PM	Afternoon
2	No Urgent Place	10AM	Morning
3	No Urgent Place	10AM	Morning
4	No Urgent Place	2PM	Afternoon
5	No Urgent Place	2PM	Afternoon
6	No Urgent Place	6PM	Evening
7	No Urgent Place	2PM	Afternoon
8	No Urgent Place	10AM	Morning
9	No Urgent Place	10AM	Morning
10	No Urgent Place	10AM	Morning

```
pd.merge(
    sql,
    sql2[['time', 'part_of_day']],
    on='time',
    how='inner'
)[['Destination', 'time', 'part_of_day']]
```

✅ Meaning of Each Part

1. `sql`

   First dataframe.

2. `sql2[['time', 'part_of_day']]`

   Selecting only the two columns from `sql2` that you need.

3. `pd.merge(..., on='time', how='inner')`

   You are performing an **INNER JOIN** between `sql` and `sql2` using the common column **time**—same as

```
[90]: #Read table_to_join
sql2 = pd.read_csv(r'C:\Users\SID Balurkar\Desktop\Full Stack Data Science\Classwork\Nov 2025\18th Nov - Data Analysis with sql vs Data Analysis with
sql2
```

[90]:

	time	part_of_day
0	2PM	Afternoon
1	10AM	Morning
2	6PM	Evening
3	7AM	Morning
4	10PM	Night

```
[89]: #SELECT a.destination,a.time,b.part_of_day FROM dataset_1 a INNER JOIN table_to_join b ON a.time=b.time
pd.merge(sql, sql2[['time', 'part_of_day']], on='time', how='inner')[['Destination', 'time', 'part_of_day']]
```

[89]:

	Destination	time	part_of_day
0	No Urgent Place	2PM	Afternoon
1	No Urgent Place	10AM	Morning
2	No Urgent Place	10AM	Morning
3	No Urgent Place	2PM	Afternoon
4	No Urgent Place	2PM	Afternoon
...	...	...	...
12679	Home	6PM	Evening
12680	Work	7AM	Morning
12681	Work	7AM	Morning
12682	Work	7AM	Morning
12683	Work	7AM	Morning

12684 rows × 3 columns

```
select destination, passanger from (select * from dataset_1 where passanger = 'Alone');
```

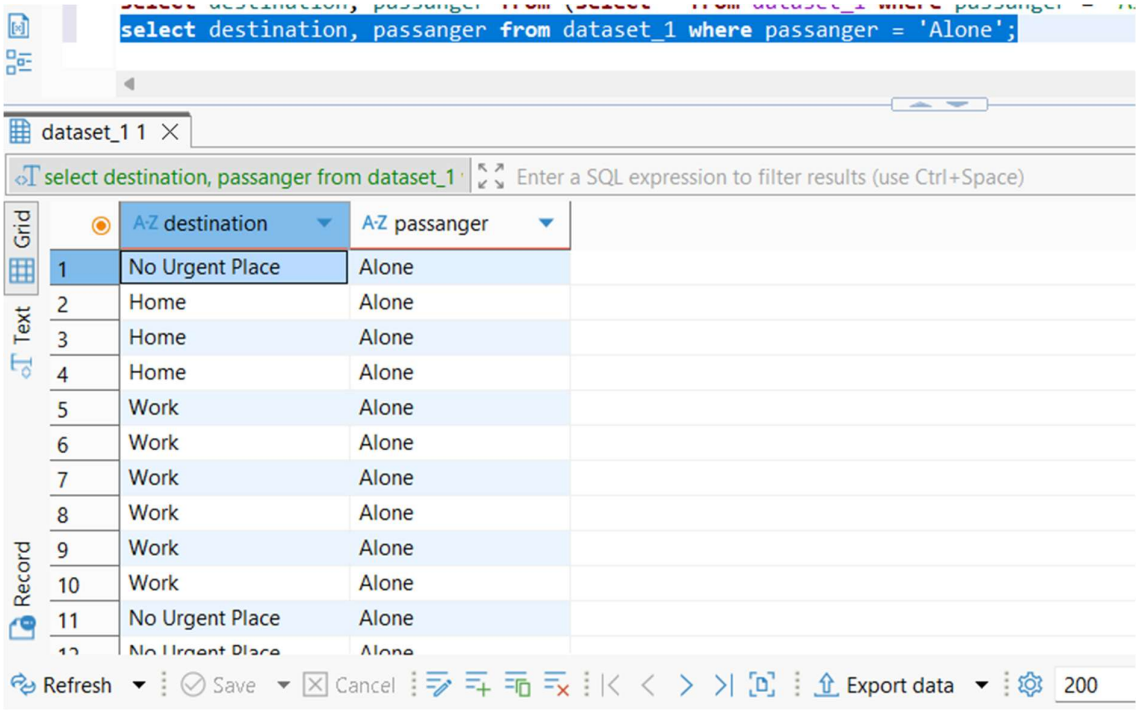
dataset_1 1

select destination, passanger from (select * fr | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text | Record

	A-Z destination	A-Z passanger
1	No Urgent Place	Alone
2	Home	Alone
3	Home	Alone
4	Home	Alone
5	Work	Alone
6	Work	Alone
7	Work	Alone
8	Work	Alone
9	Work	Alone
10	Work	Alone
11	No Urgent Place	Alone
12	No Urgent Place	Alone

Refresh | Save | Cancel | Export data | 200 | 200+



Above both queries are same.

```
]: #select destination, passanger from (select * from dataset_1 where passanger = 'Alone');
   #select destination, passanger from dataset_1 where passanger = 'Alone';
   sql[sql['passanger']=='Alone'][['Destination','passanger']]
```

]:

	Destination	passanger
0	No Urgent Place	Alone
13	Home	Alone
14	Home	Alone
15	Home	Alone
16	Work	Alone
...	...	...
12676	Home	Alone
12680	Work	Alone
12681	Work	Alone
12682	Work	Alone
12683	Work	Alone

7305 rows × 2 columns

```
select * from dataset_1 where weather like 'Sun%';
```

dataset_1 1

select * from dataset_1 where weather like 'Sı | Enter a SQL expression to filter results (use Ctrl+Space)

	destination	passanger	weather	temperature	time	coupon	expiration	gender
1	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female
3	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female
5	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female
6	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female
7	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female
11	No Urgent Place	Kid(s)	Sunny	80	2PM	Restaurant(<20)	1d	Female

```
[67]: #select * from dataset_1 where weather like 'Sun%';
      sql[sql['weather'].str.startswith('Sun')]
```

[67]:

	Destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restau
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4~8	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4~8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
12673	Home	Alone	Sunny	30	6PM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	
12676	Home	Alone	Sunny	80	6PM	Restaurant(20-50)	1d	Male	26	Single	...	1~3	4~8	
12677	Home	Partner	Sunny	30	6PM	Restaurant(<20)	1d	Male	26	Single	...	1~3	4~8	
12678	Home	Partner	Sunny	30	10PM	Restaurant(<20)	2h	Male	26	Single	...	1~3	4~8	
12683	Work	Alone	Sunny	80	7AM	Restaurant(20-50)	2h	Male	26	Single	...	1~3	4~8	

10069 rows × 27 columns

```
select distinct temperature from dataset_1 where temperature between 29 and 75;
```

dataset_1 1

select distinct temperature from dataset_1 wł | Enter a SQL expression to filter results (use Ctrl+Space)

	temperature
1	55
2	30

```
[80]: #select distinct temperature from dataset_1 where temperature between 29 and 75;
      sql[(sql['temperature'] >= 29) & (sql['temperature'] <= 75)]['temperature'].unique()
```

```
[80]: array([55, 30])
```

```
select occupation from dataset_1 where occupation in ('Sales & Related', 'Management');
```

dataset_1 1

select occupation from dataset_1 where occu | Enter a SQL expression to filter results (use Ctrl+Space)

	occupation
1	Sales & Related
2	Sales & Related
3	Sales & Related
4	Sales & Related
5	Sales & Related
6	Sales & Related
7	Sales & Related
8	Sales & Related
9	Sales & Related
10	Sales & Related
11	Sales & Related
12	Sales & Related

82]:
```
sql[sql['occupation'].isin(['Sales & Related','Management'])][['occupation']]
```

82]:

	occupation
193	Sales & Related
194	Sales & Related
195	Sales & Related
196	Sales & Related
197	Sales & Related
...	...
12679	Sales & Related
12680	Sales & Related
12681	Sales & Related
12682	Sales & Related
12683	Sales & Related

1931 rows × 1 columns